

Ein Theme für drei Shops

MDM, Borek und IMM aus einer gemeinsamen Codebasis betreiben

Stand

14. September 2026

Betrifft

MDM (DE), Borek (DE), IMM (AT)

Status

Entwurf zur Diskussion

1. Die Fragestellung

Wie können drei Shops dasselbe Theme nutzen und trotzdem individuelle Anpassungen je Marke tragen — ohne manuelles Übertragen von Code zwischen Repos?

Aufgekommen im Team am 12. September 2026. Vorgeschlagen wurde, IMM und MDM über den Markt (Shopify Markets) zu unterscheiden. Dieser Weg deckt Borek nicht ab, da Borek und MDM beide im deutschen Markt liegen.

Der heutige Zustand

Drei eigenständige Theme-Repositories mit nahezu identischem Code. Borek und IMM unterscheiden sich laut Team nur in Locales und Nuancen. Jede Änderung am Theme muss von Hand in die beiden anderen Repos übertragen werden.

**Drei Folgeprobleme**

- 1 Jede Änderung dreimal von Hand übertragen
- 2 Beim Übertragen rutschen fremde Locales mit
- 3 Die Stände drifteten auseinander, Fixes fehlen

Abbildung 1 — Ist-Zustand. Der gemeinsame Code existiert dreifach, ohne gemeinsame Quelle.

Was das Dokument liefert

Eine Zielarchitektur, in der der gemeinsame Code **einmal** existiert und die Unterschiede je Marke an definierten Stellen liegen — getrennt nach Zuständigkeit: Was Entwicklung ändert, liegt im Repository. Was Redaktion ändert, liegt im Shop.

Lesehinweis zu den Belegen

Jede Aussage ist gekennzeichnet: **BELEGT** stützt sich auf die verlinkte Shopify-Dokumentation oder auf gelesenen Code im Repository, **ANNAHME** ist plausibel, aber nicht bewiesen, **OFFEN** muss vor der Umsetzung geprüft werden. Hochgestellte Ziffern verweisen auf das Quellenverzeichnis auf der letzten Seite.

2. Befund: Warum der Markt-Ansatz hier nicht trägt

Der Vorschlag aus dem Team ist technisch nicht aus der Luft gegriffen — Shopify unterstützt marktspezifische Theme-Anpassungen tatsächlich. Er scheitert an einer anderen Stelle als zunächst vermutet.

Was Shopify Markets kann

BELEGT Ein Theme kann pro Markt angepasst werden. Sobald marktspezifische Anpassungen bestehen, legt Shopify die Datei `config/markets.json` an, die die Vererbung der Anpassungen zwischen Märkten steuert.^[4] Ein Markt ist dabei eine **Käufergruppe innerhalb eines Stores**, definiert über Land, Standort oder Company Location.^{[8][11]} In Liquid ist der aktive Markt über `localization.market` mit `handle`, `id` und `metafields` abrufbar.^[9]

Warum das die Aufgabe nicht löst

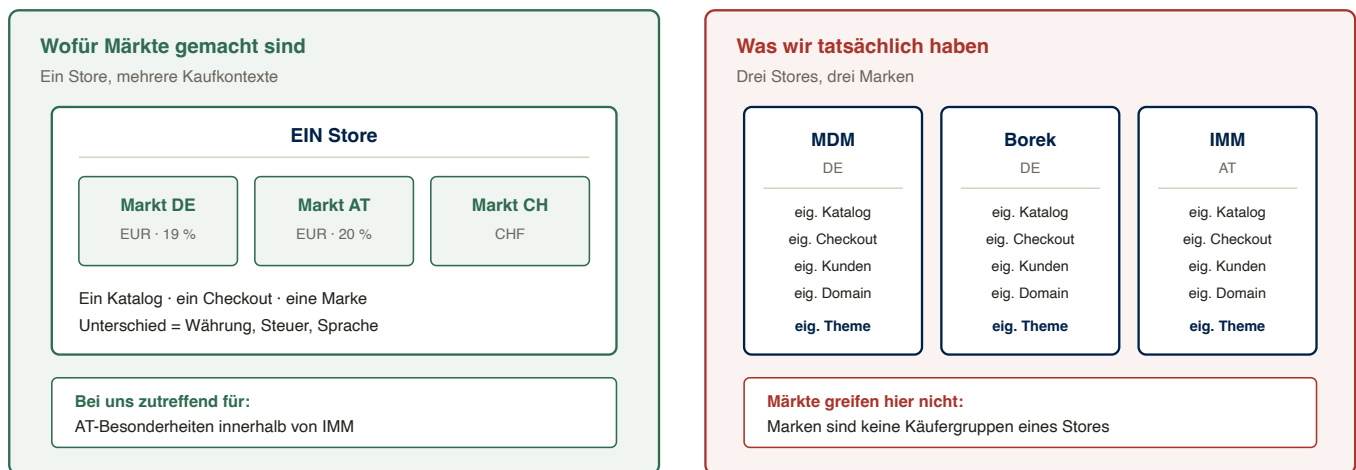


Abbildung 2 — Ein Markt trennt Kaufkontexte innerhalb eines Stores. Marken über Store-Grenzen hinweg trennt er nicht.

BELEGT Die drei Shops sind eigenständige Stores — im Datalayer-Repository als „alle 3 Mandanten Shops“ geführt. Damit greift die Markt-Logik nicht: Marktspezifische Theme-Anpassungen wirken **innerhalb** eines Stores. Über drei Stores hinweg gibt es keinen gemeinsamen Markt-Raum, in dem sie sich unterscheiden ließen.

Der Einwand aus dem Team trifft zu — aus zwei Gründen

Erstens, wie im Team benannt: Borek und MDM liegen beide im deutschen Markt. Selbst innerhalb eines Stores wäre der Markt kein Unterscheidungsmerkmal zwischen ihnen.

Zweitens, und schwerwiegender: Bei drei getrennten Stores ist die Frage „welche Marke?“ bereits durch den Store beantwortet. Ein Markt-Merkmal wäre nicht nur unzureichend, sondern überflüssig.

Wo Märkte trotzdem richtig sind

Für länderspezifische Unterschiede **innerhalb** eines Shops — österreichische Umsatzsteuer, Versandoptionen oder Zahlarten im IMM-Store. Das ist ein eigenes Thema und steht dieser Architektur nicht im Weg. **BELEGT** ^{[11][14]}

3. Zielarchitektur: Ein Code-Stand, drei Store-Zweige

Der Vorschlag beruht auf einer Trennung, die Shopify selbst vorgibt: Theme-Code gehört ins Repository, Konfiguration und Texte gehören in den Shop. Beide Seiten haben getrennte Besitzer und getrennte Änderungswege.

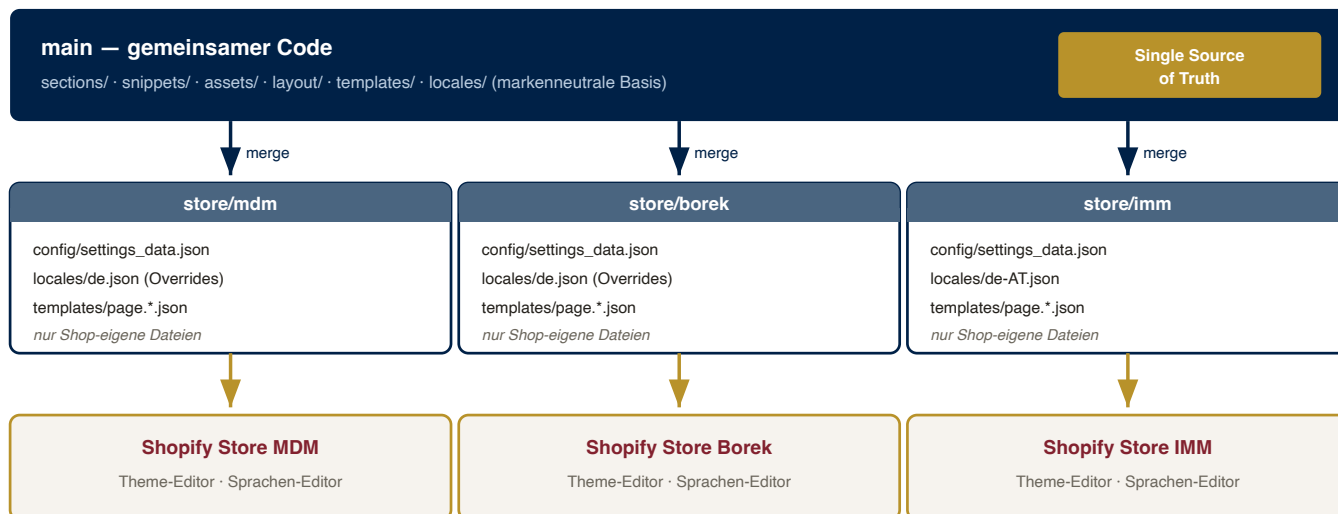


Abbildung 3 — Ein Repository, ein gemeinsamer `main`, drei dauerhafte Store-Zweige. Jede Feature-Änderung entsteht einmal auf `main` und wird in die drei Zweige gemergt.

Warum drei Zweige statt eines Branches für alle

BELEGT Die Shopify-GitHub-Integration arbeitet **bidirektional**: „Files are updated in GitHub whenever changes are made to a connected theme. This can't be disabled.“^[7] Änderungen im Theme-Editor, Code-Editor oder Sprachen-Editor werden von Shopify automatisch als Commit ins verbundene Repository zurückgeschrieben — sichtbar als `shopify[bot]`.

Hingen alle drei Stores am selben Branch, würden sie gegenseitig ihre `settings_data.json` und Locale-Werte überschreiben. Jeder Store braucht deshalb seinen eigenen Zweig als Rückschreibziel. **BELEGT** Die Dokumentation betont die Eins-zu-eins-Beziehung zwischen Theme und Branch.^[7]

Merge-Richtung ist strikt einseitig

`main` → `store/*` trägt Features in die Shops. Der Rückweg `store/*` → `main` findet **nicht** statt — dort liegen ausschließlich shopeigene Werte, die in den anderen beiden Shops falsch wären. Genau diese Einbahnstraße beseitigt das heutige Risiko, beim Übertragen fremde Locales mitzunehmen.

4. Wem gehört welche Datei

Die Architektur steht und fällt mit einer klaren Zuordnung. Shopify trennt dafür bereits sauber zwischen Code und Konfiguration.

Schicht 1 — Gemeinsamer Code Sections, Snippets, Layout, Assets, Templates · identisch in allen drei Shops · lebt auf main	Entwicklung
Schicht 2 — Branding über Theme-Einstellungen Farbschemata, Logo, Schriften, Radian in config/settings_data.json · je Store eigenständig	Redaktion im Shop
Schicht 3 — Texte über Locales Neutrale Basis auf main · Markenbegriffe interpoliert · Abweichungen im Sprachen-Editor	gemischt
Schicht 4 — Strukturunterschiede über settings.brand Nur wo ein Block existiert oder fehlt · ein Select-Setting · gekapselt in einem Snippet	Entwicklung

Abbildung 4 — Vier Schichten. Je weiter unten, desto seltener soll die Schicht gebraucht werden. Schicht 4 ist die Ausnahme, nicht die Regel.

Datei / Bereich	Zweig	Ändert wer	Begründung
sections/ snippets/ layout/ assets/	main	Entwicklung	Gemeinsamer Code, keine Markenlogik enthalten
config/settings_schema.json	main	Entwicklung	Definiert, welche Einstellungen es gibt — für alle gleich ^[3]
config/settings_data.json	store/*	Redaktion im Theme-Editor	Enthält die Werte je Shop. Wird bei jeder Editor-Änderung neu geschrieben ^[3]
locales/en.default.json locales/de.json	main	Entwicklung	Markenneutrale Basistexte, Markenname interpoliert
locales/de-AT.json	main	Entwicklung	Österreichische Sprachvariante, im IMM-Store aktiviert ^[1]
Sprachen-Editor-Overrides	store/*	Redaktion im Shop	Shopeigene Wortwahl, ohne Code-Änderung ^{[2][12]}
templates/*.json (geteilt)	main	Entwicklung	Produkt-, Kategorie- und Systemseiten mit gleicher Struktur
templates/page.<slug>.json	store/*	Redaktion im Theme-Editor	Landingpages sind je Shop verschieden
config/markets.json	store/*	Shopify automatisch	Entsteht nur bei marktspezifischen Anpassungen, z. B. in IMM ^[4]

Konfliktschutz beim Merge ist Pflicht

Beim Merge von main in einen Store-Zweig würden die shopeigenen Dateien kollidieren. Der Schutz dagegen gehört in jede store/*-Branch als .gitattributes:

```
config/settings_data.json merge=ours
templates/page/*.json      merge=ours
# einmalig je Arbeitskopie einrichten:
# git config merge.ours.driver true
```

OFFEN Der ours-Treiber muss in jeder Arbeitskopie und im CI konfiguriert sein — sonst greift die Angabe stillschweigend nicht. Das ist vor dem ersten Merge zu verifizieren.

5. Das Locale-Thema im Detail

Die Befürchtung aus dem Team — beim Übertragen rutschen unerwünschte Locales mit — ist der eigentliche Kern. Sie löst sich, wenn Texte nach ihrer Natur getrennt werden statt nach Datei.

Drei Klassen von Texten



Abbildung 5 — Entscheidungsbaum. Je weiter links ein Text landet, desto weniger Pflegeaufwand entsteht.

Klasse B in der Praxis

BELEGT Locale-Strings unterstützen Interpolation, also Variablen im Text.^[2] Der Markenname muss deshalb nirgends fest verdrahtet werden:

```
// locales/de.json – einmal auf main, gilt für alle drei Shops
{ "general": { "greeting": "Willkommen bei {{ shop }}" } }

// im Liquid
{{ 'general.greeting' | t: shop: shop.name }}

// Ergebnis je Store – ohne jede Fallunterscheidung
MDM → Willkommen bei MDM Münzhandelsgesellschaft
Borek → Willkommen bei Borek
IMM → Willkommen bei IMM
```

BELEGT `shop.name` ist in jedem Store verfügbar und liefert dessen eigenen Namen.^[10]

Klasse C: der Sprachen-Editor als Ventil

BELEGT Storefront-Locale-Dateien sind für Händler über den Shopify-Sprachen-Editor zugänglich und dort überschreibbar.^[2]

^[12] Ergänzend führt die Admin-API Theme-Locale-Inhalte als übersetzbare Ressource

`ONLINE_STORE_THEME_LOCALE_CONTENT` — Abweichungen sind also auch automatisiert pflegbar.^[13]

Österreich bekommt eine eigene Sprachdatei statt eines eigenen Repos

BELEGT Locale-Dateien folgen IETF-Sprachtags, regionale Varianten sind vorgesehen: `de.json` für Deutschland, `de-AT.json` für Österreich.^[1] Beide Dateien liegen nebeneinander auf `main` und kollidieren nicht. Der IMM-Store aktiviert Deutsch (Österreich), MDM und Borek bleiben auf Deutsch. Damit sind die AT-Nuancen im geteilten Repository abgebildet — ohne Fork, ohne Copy-Paste.

Eine Grenze, die einzuhalten ist

BELEGT Eine Locale-Datei fasst maximal 3400 Übersetzungen, ein Wert höchstens 1000 Zeichen.^[1] Bei drei Marken in einer gemeinsamen Basis ist das reichlich — solange die Datei nicht zum Ablageort für markenspezifische Varianten wird. Genau davor schützt Klasse C.

6. Zwei Wege, den Code in drei Shops zu verteilen

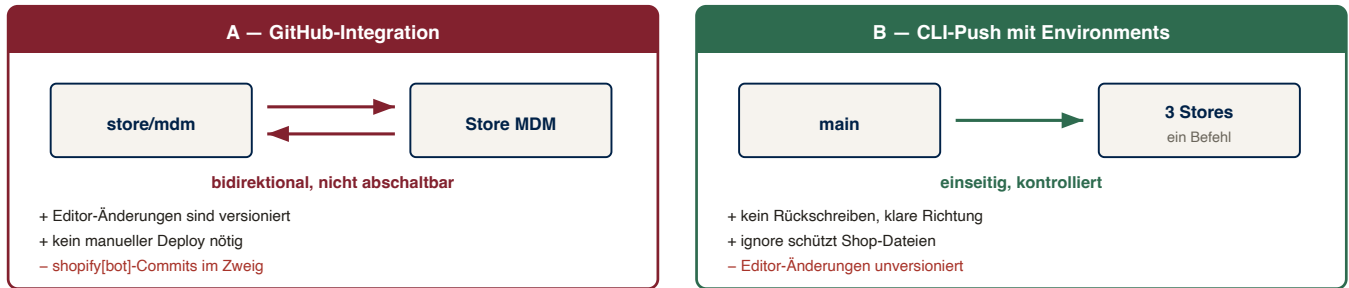


Abbildung 6 — Beide Wege sind tragfähig und lassen sich kombinieren: Variante A für den laufenden Betrieb, Variante B für gezielte Rollouts.

BELEGT Für Variante B stellt die Shopify CLI Environments bereit — mehrere Ziel-Stores in einer Datei `shopify.theme.toml`, adressierbar über `--environment`, auch mehrere gleichzeitig.^[6]

```
# shopify.theme.toml im Theme-Wurzelverzeichnis
[environments.mdm]
store = "mdm-muenze"
theme = "188378612093"
ignore = ["config/settings_data.json", "templates/page.*.json"]

[environments.borek]
store = "<borek-handle>"
theme = "<theme-id>"
ignore = ["config/settings_data.json", "templates/page.*.json"]

[environments.imm]
store = "<imm-handle>"
theme = "<theme-id>"
ignore = ["config/settings_data.json", "templates/page.*.json"]

# Rollout in alle drei mit einem Befehl
shopify theme push --environment mdm --environment borek --environment imm
```

Das ignore ist nicht optional. Ohne diesen Eintrag überschreibt jeder Push die Live-Einstellungen des Zielshops — also genau das Branding, das dort gepflegt wurde.

Empfehlung

Variante A für den laufenden Betrieb, weil sie die Arbeit der Redaktion im Theme-Editor unverändert lässt und automatisch versioniert. Variante B ergänzend für geplante Rollouts, wenn ein Feature kontrolliert in alle drei Shops gehen soll.

7. Migrationspfad

Die Reihenfolge ist bewusst so gewählt, dass der Erkenntnisgewinn vor dem Risiko kommt: Erst wissen, wie groß die Differenz wirklich ist — dann den kleinsten Shop umstellen — dann MDM.

- 1 **Differenz erheben.** Die drei Repos gegeneinander diffen und jede Abweichung einer der vier Schichten zuordnen. Ergebnis ist die Arbeitsliste — und erst sie zeigt den wahren Aufwand. **OFFEN**
- 2 **Gemeinsame Basis bestimmen.** Der Stand mit dem größten gemeinsamen Nenner wird `main`. Nach Teamauskunft liegen Borek und IMM nahezu gleichauf, MDM trägt die `mdm-` Sections. **ANNAHME** Die MDM-Basis ist der geeignete Ausgangspunkt.
- 3 **Markenbegriffe aus den Locales lösen.** Alle Treffer auf Markennamen in `locales/*.json` suchen und nach Klasse A, B oder C einsortieren. Das ist der inhaltlich aufwendigste Schritt — und der, der das Copy-Paste-Problem dauerhaft beendet.
- 4 **Store-Zweige anlegen** und je Zweig `.gitattributes` mit dem Konfliktschutz hinterlegen.
- 5 **Einen Shop pilotieren.** Borek oder IMM zuerst — nicht MDM. Der Pilot zeigt die Reibungspunkte, bevor der umsatzstärkste Shop betroffen ist.
- 6 **Übrige Shops nachziehen** und die alten Repositories archivieren, nicht löschen.

Was vor der Umsetzung zu klären ist

- **OFFEN** Laufen Borek und IMM auf derselben Theme-Basis (Hyper 1.3.3)? Falls nicht, ist der erste Schritt eine Migration, kein Zusammenführen.
- **OFFEN** Sind die drei Stores bereits per GitHub-Integration verbunden, und mit welchem Branch? Eine getrennte Verbindung lässt sich nicht wieder an denselben Branch anschließen.^[7]
- **OFFEN** Wie viele Landingpages je Shop existieren — davon hängt ab, ob `templates/page/*.json` praktikabel je Zweig geführt werden kann.

8. Zusammenfassung

Frage	Antwort
Lassen sich die Shops über den Markt unterscheiden?	Nein. Märkte trennen Käufergruppen innerhalb <i>eines</i> Stores. Bei drei getrennten Stores ist die Marke bereits durch den Store bestimmt. Der Einwand des Teams zu Borek trifft zusätzlich zu. BELEGT
Gibt es eine Store-Konfiguration?	Ja, zwei: <code>config/settings_data.json</code> trägt die Werte je Shop, und <code>shopify.theme.toml</code> beschreibt die Ziel-Stores für den Rollout. BELEGT
Wie enden die Copy-Paste-Runden?	Ein Repository mit einem <code>main</code> als gemeinsamer Quelle und drei Store-Zweigen als Rückschreibziel. Features fließen nur in eine Richtung.
Wie werden Locales sicher?	Texte nach Natur trennen: neutrale Basis auf <code>main</code> , Markennamen über <code>shop.name</code> interpoliert, abweichende Wortwahl im Sprachen-Editor des jeweiligen Shops. BELEGT
Und Österreich?	Eigene Sprachdatei <code>de-AT.json</code> neben <code>de.json</code> im selben Repository. Kein Fork nötig. BELEGT

Kern des Vorschlags in einem Satz

Der gemeinsame Code existiert genau einmal, und jede markenspezifische Abweichung bekommt einen definierten Ort — bevorzugt im Shop, wo die Redaktion sie ohne Entwicklung ändern kann.

9. Quellen

Sämtliche Quellen sind offizielle Shopify-Dokumentation, abgerufen am 14. September 2026.

- [1] Locales — Theme-Architektur**
shopify.dev/docs/storefronts/themes/architecture/locales
- [2] Storefront locale files**
shopify.dev/docs/storefronts/themes/architecture/locales/storefront-locale-files
- [3] settings_data.json**
shopify.dev/docs/storefronts/themes/architecture/config/settings-data-json
- [4] markets.json — marktspezifische Theme-Anpassungen**
shopify.dev/docs/storefronts/themes/architecture/config/markets-json
- [5] Config — Übersicht der Konfigurationsdateien**
shopify.dev/docs/storefronts/themes/architecture/config
- [6] Theme environments for Shopify CLI**
shopify.dev/docs/storefronts/themes/tools/cli/environments
- [7] Shopify GitHub integration for themes**
shopify.dev/docs/storefronts/themes/tools/github
- [8] Liquid-Objekt market**
shopify.dev/docs/api/liquid/objects/market
- [9] Liquid-Objekt localization**
shopify.dev/docs/api/liquid/objects/localization
- [10] Liquid-Objekt shop**
shopify.dev/docs/api/liquid/objects/shop
- [11] About Shopify Markets**
shopify.dev/docs/apps/build/markets
- [12] Shopify Help Center — Translating themes**
help.shopify.com/en/manual/online-store/themes/customizing-themes/language/translate-theme
- [13] Admin GraphQL — TranslatableResourceType**
shopify.dev/docs/api/admin-graphql/2026-07/enums/TranslatableResourceType
- [14] Shopify Markets für Themes**
shopify.dev/docs/storefronts/themes/markets

Interne Belege aus dem Workspace: `datalayer/README.md` (drei Mandanten-Shops), `theme/config/settings_data.json` (fünf Farbschemata), `theme/layout/mdm-theme.liquid` (Shop-Metafields im Einsatz), `.claude/rules/merchant-config.md` (Zuständigkeit für `settings_data.json`).